

Lecture 17

Operator Overloading

Overloading Unary Operators

Unary Operators

- ++ (Increment Operator)
- -- (Decrement operator)
- − (Negation operator)
- ! (Logical NOT Operator)

etc.

Overloading Unary Operators

- Unlike the operators you've seen so far, the logical not (!) operator is a unary operator, which means it only **operates on one operand**.
- It can be defined as a non-member as well as member function.

1. To overload logical NOT (**operator!**) as non-member friend function:

```
class Point {  
....  
    friend bool operator! (Point & point);  
};
```

The compiler translates "!p" to "**operator!(p)**".

2. To overload logical NOT (**operator!**) as a member function:

```
class Point {  
public:  
    bool operator!(); // this Point  
};
```

The compiler translates "!p" to "p.operator!()".

Unary Operator (logical NOT!)

```
class Point {  
private:  
    int x, y;  
public:  
    Point(int x, int y) : x(x), y(y) { }  
    bool operator! () const {  
        return (x == 0.0 && y == 0.0 ); //returns True if condition fulfills  
    }  
};
```

```
int main() {  
    Point p(0,0);  
  
    if (!p1)  
        cout << "point is set at the origin.\n";  
    else  
        cout << "point is not set at the origin.\n";  
}
```

point is set at the origin.

Unary Prefix Operator

- unary increment and decrement come in two forms: prefix (++x, --x) and postfix (x++, x--). We could do it as a non-member function as well as member function.
1. To overload ***unary prefix operator*** as a non-member friend function:

```
class Point {  
    friend Point & operator++(Point & point);  
};
```

The compiler translates "++p" to "operator++(p)".

2. To overload as a member function:

```
class Point {  
public:  
    Point & operator++(); // this Point  
};
```

The compiler translates "++p" to "p.operator++()".

Unary Postfix Operator

- Overloading postfix operators (such as `x++`, `x--`) shall be differentiated from the prefix operator (`++x`, `--x`).
- A "dummy" argument is therefore introduced to indicate postfix operation as shown below.
- Note that postfix `++` shall save the old value, perform the increment, and then return the saved value by value..
- To overload as a member function:

```
class Point {  
public:  
    Point operator++(int);    // this Point  
};
```

- The compiler translates `++p` to `p.operator++(int)`.
- And to overload as a friend function:

```
class Point {  
public:  
    friend Point operator++(Point &, int);    // this Point  
};
```

Definitions of Prefix and Postfix operator overloading

- Special syntax to distinguish pre- from post-increment
 - Implementation as member functions of the Point class

```
Point & Point::operator++() { // ++prefix, return reference of this
    ++x;
    ++y;
    return *this; // return the updated object
}
```

```
Point operator++(int) { // postfix++, return old value by value

    Point temp = *this;
    ++x; ++y;
    return temp;    // return the old value.
}
```


Usage: Prefix and Postfix overloaded operator

```
int main() {
```

```
    Point p1; // (0,0)
```

```
    cout << p1 << endl;    // displays (0,0)
```

```
    Point p2 = p1++; //increments p2
```

```
    cout<<p2<<endl; //displays (0,0)
```

```
    cout << p1 << endl;    // displays (1,1)
```

```
    cout << ++p1 << endl;    // prefix increment, displays (2,2)
```

```
}
```

Overloading [] Operator

Overloading the array subscript [] Operator

- When working with arrays, we typically use the subscript operator ([]) to index specific elements of an array:
- For example,

```
int myArr[10];  
cout<<myArr[5];
```
- However, what if we declare an array inside a class and try to access it from the main().
- We will use the public getter function to access the value at a particular index.
- And use public setter function to set array index with a value.
- While it works but it is not a natural way to access array elements.
- In addition, Range violations are common.
 - Negative indices or indices greater than the maximum cause programs to access wrong data.

Overloading the array subscript [] Operator

- We can overload the array subscript operator [] to overcome the above limitations.
- In our example, we create a class named Array.
- We overload the array subscript operator in the class Array.
- Note that array subscript operator can only be overloaded as a member function.
- We declare the array subscript operator in our class as:

```
Class Array {  
    Private:  
    ...  
    Public:  
    ..  
    int& operator[] (int i)  
    ...  
};
```

- This overloaded function always takes only one parameter, the subscript that the user places in the brackets.

Overloading the [] Operator

```
main() {  
    safeArr A(10);  
    int i;  
    i=A[5]; //read the value at A[5];  
    A[9]=3; // write to A[9]!  
    //[] operator returns a reference  
    //so we can alter the value  
}
```

- safeArr constructor creates an array dynamically.
- safeArr constructor then initializes the array.
- Functions that return references can appear on the **left hand side** of an assignment statement!

```
class safeArr {  
public:  
    safeArr(int n) : size(n) {  
        p = new int[size];  
        for (int i=0;i<size;i++) {  
            p[i] = 0;  
        }  
    }  
    ~safeArr() {delete [] p;}  
    int& operator[](int i) {  
        if (i<0 || i>=size){  
            cout<<"Index out of bounds";  
            return 0; }  
        return p[i]; }  
  
private:  
    int *p;  
    int size;  
};
```

Class Exercise

- For the safeArr class, provide the definition for the overloaded stream insertion operator

```
main() {  
    safeArr A(10);  
    cout<<"Array Elements are: "<<A;  
}
```

Class Exercise

- For the safeArr class, provide the definition for the overloaded stream insertion operator

```
friend ostream & operator<<( ostream & out, safeArr & A) {  
    for (int i = 0; i<A.size; i++)  
        out<<" "<<A.ptr[i]<<" ";  
    return out;  
}  
  
main() {  
    safeArr A(10);  
    cout<<"Array Elements are: "<<A;  
}
```

Overloading Assignment Operator =

Assignment != Construction

- “=” is the **assignment operator**
 - Assigns values to an *existing, already constructed* object
- The assignment operator (operator=) is used to copy values from one object to another already existing object.
- If you don't define the assignment operator, C++ will provide default one for you

```
Point w;           // default ctor
Point x(1, 2);     // two-ints-argument ctor
Point y(x);        // copy ctor
Point z = w;       // copy ctor. Note= in declaration
y = x;             // assignment operator
```

- Difference is that the copy constructor is making a new object while the assignment operator is dealing with an existing object (the one on the left side of the operator).

Assignment != Construction

- “=” is the **assignment operator**
 - Assigns values to an *existing, already constructed* object
- If you don't define the assignment operator, C++ will provide default one for you
 - But It will do a *shallow* copy of all of the fields (*i.e.* member variables) of your class
 - Same problem as default copy constructor (covered already in **lecture#11**)

```
Point w;           // default ctor
Point x(1, 2);     // two-ints-argument ctor
Point y(x);        // copy ctor
Point z = w;       // copy ctor. Note= in declaration
y = x;             // assignment operator
```

- Difference is that the copy constructor is making a new object while the assignment operator is dealing with an existing object (the one on the left side of the operator).

Default Assignment '=' operator

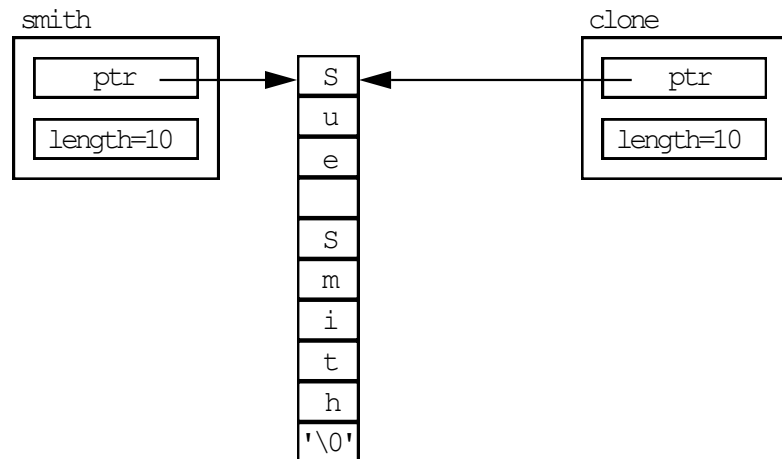
```
name smith("sue smith"); //constructor
```

```
name clone; //create a new object with default constructor
```

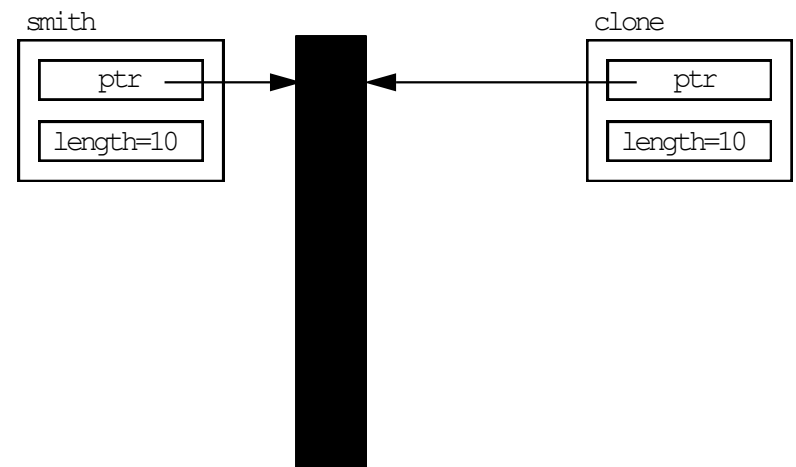
```
clone = smith; //Default '=' assignment operator
```

- following **shallow copy** due to default assignment operator is disastrous!
- If anyone object calls destructor, the contents in the memory will be deleted
 - We need to overload the '=' operator for the **deep copy**
 - Any dynamic memory pointed to by the data members is duplicated and the contents of that memory is copied (via overloaded assignment operator)

Memory Layout after using **default assignment operator**



Memory Layout after one of the object goes out of scope



Overloading = Operator

```
class name {  
    public:  
    name(const char* name_ptr= ""); //default constructor  
    ~name(){ delete [] ptr;}        //destructor  
  
    void operator = (const name & ); //assignment  
    void display(){cout<<ptr<<endl;  
private:  
    char* ptr; //pointer to name  
    int length; //length of name including nul char  
};
```

- What is the definition of the class constructor??
- What is the definition of the default assignment operator??

Overloading = Operator

```
class name {  
    public:  
    name(const char* name_ptr= ""); //default constructor  
    ~name(){ delete [] ptr; } //destructor  
  
    void operator = (const name & ); //assignment  
    void display(){cout<<ptr<<endl;  
private:  
    char* ptr; //pointer to name  
    int length; //length of name including nul char  
};
```

```
name::name(const char* name_ptr) {  
    length = strlen(name_ptr);  
    ptr = new char[length];  
    strcpy(ptr, name_ptr);  
}
```

- If you do not provide overloaded assignment operator, the default one will look like this:

```
//Default Assignment operator  
void operator = (const name & n2) {  
    length = n2.length;  
    ptr = n2.ptr;  
}
```

Overloading = Operator

```
class name {
public:
name(const char* name_ptr= "");           //default constructor
~name(){ delete [] ptr;}                 //destructor

void operator = (const name & ); //assignment
    void display(){cout<<ptr<<endl;}
private:
    char* ptr; //pointer to name
    int length; //length of name including nul char
};

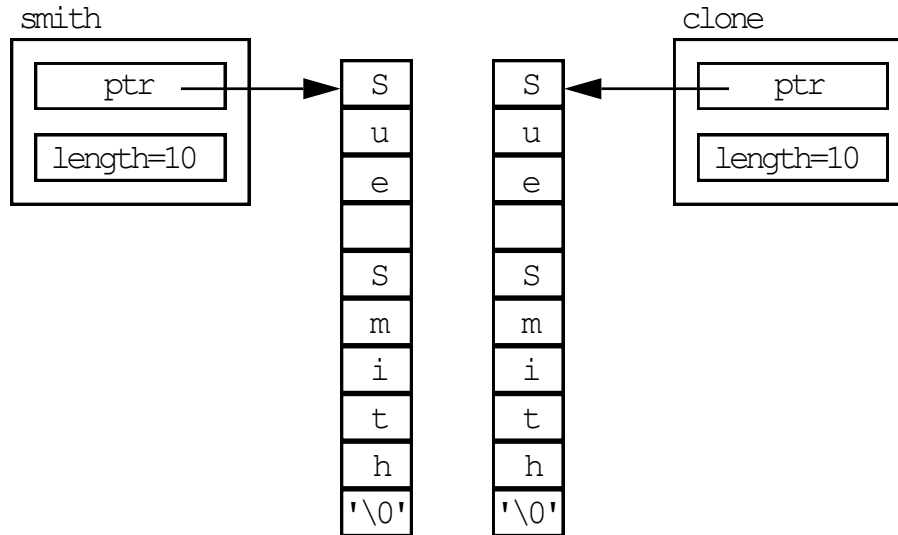
//Customized assignment operator for deep copying

void operator = (const name & n2) {

delete [] ptr; //deallocate any dynamic memory
length = n2.length;
ptr = new char [length];
strcpy(ptr,n2.ptr); //copies contents that ptr is pointing to
}
```

Overloading Assignment '=' operator

- After overloading assignment '=' operator.
- Now if one object calls destructor, the other object is not affected.



Class exercise: Overloading Assignment '=' operator

- Rewrite the class SafeArray but now also put the copy constructor and overloaded assignment '=' operator functionality.

```
main() {  
    safeArr A(10);  
    int i;  
    i=A[5]; //read the value at A[5];  
    A[9]=3; // write to A[9]!  
  
    SafeArr b;  
    b = a;  
  
    B[5] = 55;  
  
    cout<<"Array B: "<<B;  
  
}
```